

# **APLICACIONES MOVILES**

## **LABORATORIO 3B**

### **La misma app, tres veces: el estado se muda**

## Módulo 2 — Desarrollo nativo · Guía autónoma · Tiempo estimado: 90 minutos · Segunda parte de tres (3A → 3B → 3C)

**El problema de hoy:** mismo producto, otra arquitectura. Vas a refactorizar AppTeca para que el estado deje de vivir en la pantalla — y el giro que ayer rompía tu app hoy no pueda tocarla.

**Qué necesitás antes de empezar:** tu proyecto **AppTeca** del 3A, funcionando completo (lista, detalle, favoritos, buscador, modo "solo favoritas"). No vas a crear un proyecto nuevo: **este laboratorio es un refactor** — la operación profesional más común del mundo real: mejorar una app existente sin romperla y sin reescribirla. Tu ficha comparativa, a mano: hoy se llena la columna B.

**Teoría que acompaña:** el bloque de la evolución de las formas de desarrollar — el segundo eje: la organización, MVVM y el ViewModel. Y la suscripción a eventos: el estado observable que hoy vas a construir es el patrón observador de esa teoría, trabajando para vos.

### El destino

Al terminar, AppTeca va a hacer **exactamente lo mismo que hace hoy** — para el usuario, nada cambió. Todo el cambio es interno: el estado y la lógica de filtros van a vivir en un **ViewModel** que sobrevive a la recreación de la pantalla; la Activity va a quedar reducida a lo que el patrón dice que debe ser (mostrar y avisar); el sincronizador manual —vos, llamando `aplicarFiltros()` en cada esquina— va a ser reemplazado por **una suscripción**; y una línea de código célebre del 3A va a ser **eliminada** en una pequeña ceremonia. La prueba final: el giro que apagaba el modo "solo favoritas" ya no va a poder apagarlo.

### Eta 1 — El plano de la mudanza

*Ningún refactor serio empieza tocando código: empieza entendiendo qué hay y decidiendo qué se mueve. Esta etapa es con papel.*

**1.1 — El inventario del estado.** Abrí tu `MainActivity` del 3A y hacé el inventario por escrito: completá esta tabla con **cada pieza de estado** de la pantalla principal, dónde vive hoy, y qué le pasó en los experimentos de la Eta 5 del 3A:

| Pieza de estado                                         | ¿Dónde vive hoy? | ¿Sobrevivió al giro? | ¿A la muerte del proceso? |
|---------------------------------------------------------|------------------|----------------------|---------------------------|
| El texto de búsqueda                                    |                  |                      |                           |
| El modo "solo favoritas"                                |                  |                      |                           |
| Los favoritos<br>( <code>esFavorita</code> de cada app) |                  |                      |                           |
| La lista visible (filtrada)                             |                  |                      |                           |

✓ *Verificación: tabla completa. Las respuestas que tenés que haber encontrado: el texto vive en una vista con id (sobrevivió al giro por el sistema); el modo vive en una variable de la Activity (murió con el giro — el bug del 3A); los favoritos viven en el singleton `Catalogo` (sobrevivieron al giro, murieron con el proceso); la lista visible no vive en ningún lado — se recalcula. Si algo no te*

*coincide, releé la Etapa 5 del 3A antes de seguir: este diagnóstico es el fundamento de todo lo que viene.*

**1.2 — El plano.** Con el inventario a la vista, la decisión de mudanza — leela y contrastala con tu tabla:

- **Se muda al ViewModel:** el modo `soloFavoritas`, el texto de búsqueda (su *copia de trabajo* — la vista sigue mostrándolo, pero la verdad pasa a estar en el ViewModel), la lista visible (que deja de recalcularse "cuando alguien se acuerda" y pasa a *publicarse* cada vez que cambia), y la función `aplicarFiltros()` entera — que es **lógica de presentación**, no de pantalla.
- **Se queda donde está:** las vistas y el layout (no cambian ni una línea), el adaptador (cambia una línea), la navegación por Intent, y el `Catalogo` (sigue siendo la fuente de datos — sus problemas son de *persistencia*, y ese es otro capítulo de la teoría, no este refactor).

La regla que ordena el plano, para tu registro: **al ViewModel va el estado de la pantalla y la lógica que lo transforma; en la Activity queda solo lo que necesita una pantalla viva** — mostrar, escuchar toques, navegar.

## Etapa 2 — Los cimientos: el ViewModel existe y sobrevive

*Antes de mudar nada, la casa nueva: una clase ViewModel conectada a la Activity — y la primera prueba de que el sistema la trata distinto.*

**2.1 — Las dependencias.** En `build.gradle.kts` (Module :app), dentro del bloque `dependencies`, agregá (y sincronizá):

```
implementation("androidx.activity:activity-ktx:1.9.3")
implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.7")
implementation("androidx.lifecycle:lifecycle-livedata-ktx:2.8.7")
```

[△ VERIFICAR: versiones vigentes al preparar el dictado] Son las piezas de Jetpack para esta era: el ViewModel, LiveData (el contenedor observable clásico de la era Views — enseguida lo conocés), y la extensión que permite obtener el ViewModel con una línea.

✓ *Verificación: sincronización sin errores.*

**2.2 — La clase, vacía pero viva.** Nuevo archivo Kotlin, tipo File, `AppTecaViewModel`:

```
package com.example.appteca

import android.util.Log
import androidx.lifecycle.ViewModel

class AppTecaViewModel : ViewModel() {
    init {
        Log.d("VIDA", "ViewModel → creado (${hashCode()}")
    }

    override fun onCleared() {
        Log.d("VIDA", "ViewModel → onCleared (destruido de verdad)")
    }
}
```

La instrumentamos desde el nacimiento — el `hashCode()` en el log es la "cédula de identidad" de la instancia: va a ser la prueba de la supervivencia.

**2.3 — La conexión, y por qué no es un constructor.** En `MainActivity`, arriba de todo (junto al `adapter`):

```
private val vm: AppTecaViewModel by viewModels()
```

(Import: `androidx.activity.viewModels`.) Leé esa línea con atención, porque su forma **es** la lección: no escribiste `AppTecaViewModel()` — no lo construís vos. El `by viewModels()` le pide la instancia **al sistema**, y el sistema decide: si es la primera vez, la crea; si la Activity está siendo *recreada* (un giro), **le devuelve la que ya existía** — viva, con todo su estado adentro. Un ViewModel construido a mano sería una variable más de la Activity y moriría con ella; el pedido al sistema es lo que lo pone fuera del alcance de la recreación. Es el "control prestado" de siempre, ahora jugando a favor.

**2.4 — La primera prueba de supervivencia.** Ejecutá, mirá el Logcat (filtro `tag:VIDA` — tu instrumentación del Lab 2 sigue ahí y ahora tiene un actor nuevo), y **girá el dispositivo**:

```
ViewModel → creado (127890)
Main → onCreate
...
[giro]
Main → onDestroy
Main → onCreate
...
```

✓ Verificación: en el giro, la Activity muere y renace (sus logs lo cuentan) — pero no hay un segundo "ViewModel → creado": el `hashCode` sigue siendo el del nacimiento, porque la instancia es la misma. Y `onCleared` no apareció nunca: el ViewModel se destruye recién cuando la pantalla se va de verdad (cerrá la app con atrás desde la principal y lo vas a ver aparecer). Acabás de presenciar el hecho técnico sobre el que se construye todo este laboratorio: hay un lugar que el giro no puede tocar.

## Problemas frecuentes de la Etapa 2

- **"Unresolved reference: viewModels":** falta la dependencia `activity-ktx` o el import correcto (`androidx.activity.viewModels` — hay otro parecido para Fragments que no es este).
- **Veo "ViewModel → creado" dos veces tras el giro:** lo estás construyendo a mano en algún lado (`AppTecaViewModel()`) — buscá y eliminá; la única vía es el `by viewModels()`.
- **`onCleared` aparece en el giro:** mismo síntoma que el anterior — una instancia construida a mano muere con la Activity. El log te está mostrando exactamente el error que la línea `by viewModels()` evita.

## Etapa 3 — La mudanza: el estado y su lógica se van al ViewModel

La casa está construida y probada. Ahora se muda el estado — y con él, la función que lo transforma. La novedad técnica de la etapa: el estado no se muda como variables comunes, sino como estado observable.

**3.1 — LiveData: el contenedor que avisa.** Antes de escribir, el concepto. Si el ViewModel guardara la lista visible en una variable común, la Activity tendría que *preguntarle* cada vez ("¿cambió algo?") —

y estaríamos de vuelta en el sincronizador manual, solo que mudado de casa. La pieza que cambia el juego es **LiveData**: un contenedor de un valor que, además de leerse, **se observa** — quien se suscribe recibe el valor actual y cada cambio futuro, automáticamente. Es el patrón observador de la teoría, empaquetado por la plataforma para la era Views, y con una propiedad extra que vas a ver trabajar en la prueba final: es consciente del ciclo de vida — solo notifica a pantallas vivas, y al suscriptor nuevo le entrega el último valor al instante.

**3.2 — El ViewModel completo.** Reemplazá el contenido de `AppTecaViewModel` por su versión final — leeelo entero: es la mudanza consumada:

```
package com.example.appteca

import android.util.Log
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class AppTecaViewModel : ViewModel() {

    // — El estado interno (antes: variables de la Activity) —
    private var query = ""
    private var soloFavoritas = false

    // — El estado publicado (antes: no existía — cada listener redibujaba) —
    private val _listaVisible = MutableLiveData<List<App>>()
    val listaVisible: LiveData<List<App>> = _listaVisible

    private val _modoSoloFavoritas = MutableLiveData(false)
    val modoSoloFavoritas: LiveData<Boolean> = _modoSoloFavoritas

    init {
        Log.d("VIDA", "ViewModel → creado (${hashCode()}")
        aplicarFiltros()
    }

    // — Los eventos que la pantalla puede avisar —
    fun buscar(texto: String) {
        query = texto.trim()
        aplicarFiltros()
    }

    fun alternarModo() {
        soloFavoritas = !soloFavoritas
        aplicarFiltros()
    }

    fun alternarFavorita(app: App) {
        app.esFavorita = !app.esFavorita
        aplicarFiltros()
    }

    // — LA función, mudada casi textual — con una diferencia decisiva —
    private fun aplicarFiltros() {
        var lista: List<App> = Catalogo.apps
        if (query.isNotEmpty()) lista = lista.filter {
            it.nombre.contains(query, true) || it.categoria.contains(query,
true)
        }
        if (soloFavoritas) lista = lista.filter { it.esFavorita }
        _listaVisible.value = lista // publica — no dibuja
        _modoSoloFavoritas.value = soloFavoritas
    }
}
```

```

    override fun onCleared() {
        Log.d("VIDA", "ViewModel → onCleared (destruido de verdad)")
    }
}

```

Tres cosas para leer antes de seguir. **Uno:** el par privado/público (`_listaVisible` mutable adentro, `listaVisible` de solo lectura afuera) es el encapsulamiento del patrón — la pantalla puede *observar* el estado, pero solo el ViewModel puede *cambiarlo*: los datos fluyen hacia abajo, los eventos suben como llamadas (`buscar`, `alternarModo...`), nunca al revés. **Dos:** `aplicarFiltros()` es *tu misma función* del 3A, casi carácter por carácter — pero perdió sus dos últimas líneas: **ya no toca vistas**. No conoce al adaptador, no conoce al botón: calcula y **publica**. Quién escucha y qué hace con eso ya no es problema suyo. **Tres:** esta clase no importa nada de Android UI — ni Activity, ni View, ni adaptadores. Guardá ese dato para la pregunta 5 de la ficha.

✓ Verificación: compila sin errores. La Activity todavía no lo usa — está rota a medias si la ejecutás; la conexión es la próxima etapa.

## Etapa 4 — La suscripción: el sincronizador se jubila

Del otro lado de la mudanza, la Activity queda liviana: muestra, escucha toques, avisa. Y una línea célebre se elimina con honores.

**4.1 — La Activity final.** Reemplazá tu `MainActivity` completa — compará mentalmente con la del 3A mientras la leés:

```

class MainActivity : AppCompatActivity() {

    private val vm: AppTecaViewModel by viewModels()
    private lateinit var adapter: AppAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        adapter = AppAdapter(emptyList(),
            onAppClick = { app ->
                val intent = Intent(this, DetalleActivity::class.java)
                intent.putExtra("appId", app.id)
                startActivity(intent)
            },
            onFavoritoClick = { app -> vm.alternarFavorita(app) })

        val rv = findViewById<RecyclerView>(R.id.rvApps)
        rv.layoutManager = LinearLayoutManager(this)
        rv.adapter = adapter

        findViewById<EditText>(R.id.etBuscar).addTextChangedListener { texto ->
            vm.buscar(texto.toString())
        }

        findViewById<Button>(R.id.btnSoloFav).setOnClickListener {
            vm.alternarModo()
        }
    }
}

```

```

// — LAS suscripciones: el único lugar donde el estado toca la pantalla
—
vm.listaVisible.observe(this) { lista ->
    adapter.actualizarLista(lista)
}
vm.modoSoloFavoritas.observe(this) { activo ->
    findViewById<Button>(R.id.btnSoloFav).text =
        if (activo) "★ Solo favoritas" else "☆ Todas"
}
}
}

```

Mirá lo que quedó — y lo que no. El adaptador nace con `emptyList()`: la Activity **ya no sabe qué se muestra** — lo que se muestra llega por la suscripción, y llega solo (LiveData entrega el valor actual al suscribirse: por eso no hace falta ninguna "llamada inicial"). Los listeners quedaron reducidos a una línea cada uno: *avise al ViewModel* — no calculan, no filtran, no redibujan. Cada flecha va en una sola dirección: **los toques suben como llamadas; el estado baja por la suscripción; y el punto de encuentro entre estado y pantalla es uno solo**. En el 3A, olvidarte de llamar `aplicarFiltros()` en un camino era un bug silencioso; acá ese bug es *inexpresable* — no hay nada que acordarse de llamar.

**4.2 — La ceremonia.** Falta algo, y lo vas a borrar vos. En tu 3A había un método que ya no copiaste (si lo copiaste, este es su momento): el `onResume` con su comentario — *"por si algo cambió mientras no miraba"*. Ese método era la confesión del modelo anterior: el sincronizador admitiendo que no podía saber cuándo sincronizar, y redibujando todo *por las dudas*. Borralo, y lee una vez más lo que estás borrando: **el código defensivo existe donde la arquitectura no garantiza; cuando la garantía llega, el código defensivo se va**. Ejecutá.

✓ *Verificación: AppTeca funciona exactamente igual que ayer — lista, detalle, estrellas, buscador, modo, todo. Para el usuario no cambió nada; para vos cambió todo. Si algo no anda, los Problemas frecuentes te esperan abajo.*

#### Problemas frecuentes de la Etapa 4

- **La lista arranca vacía y no se llena:** la suscripción `observe` quedó sin escribirse, o el `init` del ViewModel no llama `aplicarFiltros()` — sin publicación inicial no hay primer valor que entregar.
- **"Cannot add the same observer with different lifecycles":** pusiste el `observe` en un lugar que se ejecuta más de una vez (¿un listener?) — las suscripciones van una sola vez, en `onCreate`.
- **El botón no cambia de texto:** te suscribiste a `listaVisible` pero no a `modoSoloFavoritas` — son dos publicaciones, dos suscripciones.
- **Marcá favorita y la estrella no cambia:** el `onFavoritoClick` tiene que llamar a `vm.alternarFavorita(app)` — si quedó el código viejo (mutar + notify), anda "de casualidad" y por el camino equivocado: el estado cambió por afuera del ViewModel.

## Etapa 5 — La prueba de fuego — y tu ficha

*El refactor está hecho y la app funciona igual. "Igual" era el requisito para el usuario — ahora toca demostrar en qué es mejor. Los experimentos son exactamente los del 3A, para comparar de verdad.*

**5.1 — El giro, revancha.** El experimento que rompió al 3A, paso a paso idéntico: marcá dos o tres favoritas, escribí algo en el buscador, activá **★ Solo favoritas**, y rotá.

✓ Verificación: todo sobrevivió. El botón sigue en "★ Solo favoritas", la lista sigue filtrada, la búsqueda sigue aplicada. Ahora explícalo con el Logcat en la mano: la Activity murió y renació (sus logs lo cuentan) — pero el ViewModel es el mismo (`hashCode` intacto, sin "creado" nuevo), y la Activity recién nacida, al suscribirse, recibió el último valor publicado al instante — la propiedad de LiveData de la Etapa 3, cerrando el círculo. Nadie "restauró" nada: el estado nunca estuvo en peligro, porque nunca vivió en lo que murió. El bug del 3A no se arregló — se volvió imposible de escribir.

**5.2 — El límite, confesado y verificado.** La teoría dice que el ViewModel sobrevive a la recreación, no a la muerte del proceso — y en esta materia los límites se verifican, no se creen. El arma de siempre: favoritas marcadas, modo activo, Home, `adb shell am kill com.example.appteca`, volver desde recientes.

✓ Verificación: modo apagado, favoritos en blanco — el ViewModel también murió: era memoria del proceso, como el singleton (el Logcat lo confirma: hay un "ViewModel → creado" nuevo, con otro `hashCode`). El mapa de muertes de tu inventario de la Etapa 1 suma su fila final: el ViewModel sobrevive al giro pero no al proceso. Contra la muerte del proceso hay dos defensas — el rescate del estado chico (existe un mecanismo del ViewModel para eso: `SavedStateHandle`, tu desafío 2) y la persistencia real de los datos (el capítulo que la teoría tiene pendiente) — y el TP va a necesitar la segunda.

**5.3 — La grieta, revisitada.** Un último experimento honesto. En el 3A, marcar favorita desde el detalle dejaba la lista desactualizada, y lo parchaste con el `onResume` — que hoy borraste con honores. Repetí el experimento: entrá al detalle de una app sin estrella, marca favorita con el botón, volvé.

✓ Verificación: la lista está desactualizada otra vez — la grieta volvió con la eliminación del parche. Y esta es la lección fina del laboratorio: el ViewModel no arregló esto, porque esto nunca fue un problema de supervivencia — es un problema de fuente de verdad: `DetalleActivity` muta el `Catalogo` por afuera, y el ViewModel de la lista no tiene forma de enterarse. La solución de fondo tiene nombre en la teoría — una fuente de datos única y observable, de la que todas las pantallas deriven — y la vas a construir cuando el catálogo viva en una base de datos real. Por hoy, el parche pragmático: `override fun onResume() { super.onResume(); vm.refrescar() }` con un método público de una línea en el ViewModel que llame a `aplicarFiltros()`. Sí: el `onResume` vuelve — pero leé la diferencia: ya no redibuja "por las dudas"; le pide a la fuente que re-derive. El día que la fuente sea observable, esta línea se borra sola.

**5.4 — La ficha comparativa: columna B.** Con los tres experimentos hechos, llená la segunda columna de tu ficha. Dos preguntas merecen atención especial. La 4 (¿qué costaría agregar un campo?): pensá dónde tocarías — ¿cuántos de esos lugares están en el ViewModel y cuántos en la vista? La 5 (¿qué se puede probar sin emulador?): abrí `AppTecaViewModel` y mirá sus imports — no hay una sola clase de interfaz. Esa clase se puede instanciar en un test común de JUnit, llamarle `buscar("spo")` y afirmar sobre `listaVisible.value`. En el 3A, la lógica de filtros vivía dentro de una Activity que solo el sistema podía crear; hoy es una clase que cualquier test instancia. La testeabilidad no se agregó: quedó de regalo por poner cada cosa en su capa.

✓ Verificación: columna B completa, con la A al lado para comparar.

## Desafíos opcionales

*Sin guía paso a paso — de eso se trata.*

1. **El test que ayer era imposible.** Escribí el test de JUnit de la lógica de filtros: instanciá el ViewModel, simulá búsquedas y alternancias, afirmá sobre el valor publicado. Vas a necesitar una regla de test para LiveData (`InstantTaskExecutorRule` — investigá por qué) y vas a descubrir el detalle del `Catalogo` compartido entre tests. Ambos obstáculos son lecciones.
2. **Sobrevivir también al proceso.** Investigá `SavedStateHandle`: el constructor del ViewModel puede recibirlo, y lo que guardes ahí sobrevive a la muerte del proceso (el sistema lo rescata como hacía con las vistas con id). Mudá `soloFavoritas` y `query` ahí, repetí el experimento 5.2, y verificá qué sobrevive ahora — y qué sigue sin sobrevivir (los favoritos: datos, no estado de pantalla — cada cosa en su defensa).
3. **La semilla de los Fragments.** En el código profesional Views+MVVM vas a encontrar este mismo patrón con **Fragments** dentro de una Activity única (la segunda era de la teoría de navegación). Investigá el ViewModel compartido (`activityViewModels()`): dos Fragments hermanos pueden observar *el mismo* ViewModel. ¿Cuál de los problemas que este laboratorio **no** resolvió se arreglará con eso — y por qué acá, con dos Activities, no se puede?

## Checklist de salida

- ☐ AppTeca refactorizada: mismo comportamiento visible, estado y lógica en `AppTecaViewModel`, Activity reducida a mostrar/escuchar/avisar
- ☐ Vi la supervivencia del ViewModel en el Logcat (mismo `hashCode` tras el giro) y sé explicar por qué `by viewModels()` no es un constructor
- ☐ El giro ya no apaga el modo — y sé explicar por qué el bug se volvió *inexpresable*
- ☐ Verifiqué el límite: el ViewModel muere con el proceso, y sé cuáles son las dos defensas
- ☐ Redescubrí la grieta del detalle y sé por qué el ViewModel no la arregla (supervivencia ≠ fuente de verdad)
- ☐ Columna B de mi ficha, completa y comparada con la A
- ☐ (Opcional) El test imposible · `SavedStateHandle` · la semilla de los Fragments

**Adónde va esto:** la arquitectura ya es la definitiva — MVVM con estado observable es exactamente lo que vas a usar en el trabajo práctico. Lo que falta cambiar es la otra mitad de la historia: la interfaz. Tu XML, tu adaptador con sus cuatro piezas, tus `findViewById` — toda esa ceremonia tiene un reemplazo donde la pantalla se *describe* en Kotlin y se redibuja sola cuando el estado cambia. Proyecto nuevo, mismo producto, tercera forma: **Compose**. Y una pista de lo que vas a sentir: tu ViewModel de hoy va a mudarse casi sin tocarlo — porque ya habla el idioma que Compose espera.